

Day 79: Seaborn Fundamentals

1. Day Number

Day 79

2. Topic Name

Statistical Visualization with Seaborn

Today you will learn how to create simple charts using **Seaborn**, especially when your data is already stored in a **Pandas DataFrame**.

3. Connection to Yesterday

Yesterday, in **Day 78**, you learned the fundamentals of **Matplotlib** and created basic charts.

You worked with ideas such as:

- figure and axes
- line charts
- bar charts
- chart titles
- axis labels

Today you will build on that knowledge.

Matplotlib → basic plotting foundation

Seaborn → convenient **DataFrame**-based statistical visualization

Seaborn actually uses Matplotlib underneath, so the concepts you learned yesterday are still useful.

4. Important Topics

Seaborn

Seaborn is a Python library for creating data visualizations.

It works especially well with **Pandas DataFrames**.

A common import is:

```
import seaborn as sns
```

You will often also see:

```
import matplotlib.pyplot as plt
```

Matplotlib is still useful for things such as showing the chart and adding some labels.

DataFrame-based plotting

Suppose your DataFrame contains:

	hours	score
0	1	45
1	2	55
2	3	65

With Seaborn, you can tell it:

- which DataFrame contains the data
- which column should be on the x-axis
- which column should be on the y-axis

Conceptually:

```
sns.some_plot(  
    data=df,  
    x="hours",  
    y="score"  
)
```

This is one reason Seaborn is convenient when working with Pandas.

Bar plot

A **bar plot** uses rectangular bars to compare values between categories.

For example:

Product A	■ ■ ■ ■
Product B	■ ■ ■ ■ ■ ■
Product C	■ ■

It can be useful for comparing things such as:

- sales by product
- marks by subject
- customers by city

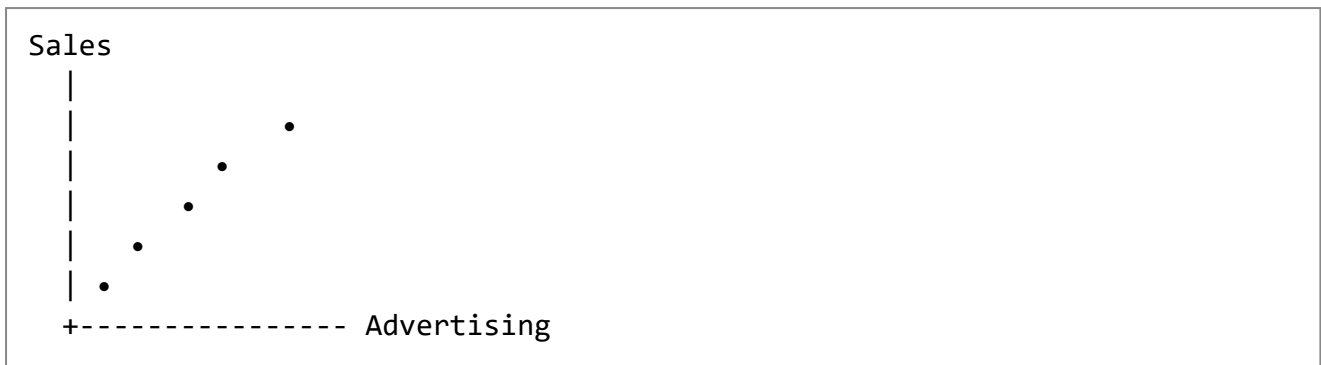
A simple Seaborn function is:

```
sns.barplot(...)
```

Scatter plot

A **scatter plot** displays individual data points.

For example:



Scatter plots are useful when you want to examine whether **two numerical variables appear related**.

For example:

- advertising spend vs sales
- study hours vs exam score
- temperature vs ice-cream sales

A common Seaborn function is:

```
sns.scatterplot(...)
```

Distribution plot idea

Sometimes we want to understand **how values are distributed**.

Suppose exam scores are:

```
45, 52, 55, 58, 60, 60, 61, 70, 85
```

A distribution visualization can help answer questions such as:

- Where are most values?
- Are values spread out?
- Are some values unusually high or low?

For now, simply understand the idea.

A common beginner-friendly Seaborn function for viewing a distribution is:

```
sns.histplot(...)
```

You do **not** need advanced statistical distribution plots yet.

5. Foundational Notes

Seaborn works well with Pandas

You have already learned DataFrames.

Suppose:

```
import pandas as pd

data = {
    "day": ["Mon", "Tue", "Wed"],
    "customers": [20, 35, 28]
}

df = pd.DataFrame(data)
```

Seaborn can directly use column names such as:

```
day
customers
```

instead of requiring you to manually separate every list.

data=

This tells Seaborn which DataFrame to use.

Concept:

```
data=df
```

x=

This tells Seaborn which column should appear horizontally.

Example:

```
x="day"
```

y=

This tells Seaborn which column should appear vertically.

Example:

```
y="customers"
```

So a Seaborn command often follows this mental pattern:

```
Choose chart
↓
Give DataFrame
↓
Choose x column
↓
Choose y column
```

6. Seaborn vs Matplotlib

Think of them like this:

Matplotlib	Seaborn
General-purpose plotting library	Visualization library built on Matplotlib
Gives detailed control	Convenient defaults
Often works directly with lists/arrays	Very convenient with DataFrames
Foundation for many Python plots	Designed especially for statistical/data visualization

For example, with Matplotlib you might think:

```
Plot these x values against these y values.
```

With Seaborn, you often think:

```
Use this DataFrame.  
  
Put this column on x.  
  
Put this other column on y.
```

Neither replaces the other.

In real Python projects, you will often see **Seaborn and Matplotlib used together**.

7. Easy Example

Suppose we have information about students:

```
import pandas as pd  
  
data = {  
    "study_hours": [1, 2, 3, 4],  
    "score": [45, 55, 68, 78]  
}  
  
df = pd.DataFrame(data)
```

The DataFrame looks approximately like:

```
   study_hours  score  
0             1     45  
1             2     55  
2             3     68
```

3	4	78
---	---	----

We could create a scatter plot using the basic pattern:

```
sns.scatterplot(  
    data=df,  
    x="study_hours",  
    y="score"  
)
```

Each row becomes approximately **one point** on the chart.

For example:

```
study_hours = 1, score = 45
```

becomes a point representing:

```
(1, 45)
```

This makes relationships between numerical columns much easier to see.

8. Problem Statement

Create a small Pandas DataFrame containing two columns:

```
advertising_spend  
sales
```

For example, your data might conceptually represent:

Advertising Spend	Sales
100	1200
200	1800
300	2500
400	3100
500	3900

Your program should:

1. Import the required libraries.
2. Create a small DataFrame.
3. Store several advertising-spend values.
4. Store corresponding sales values.
5. Create a **Seaborn scatter plot**.
6. Place advertising spend on the **x-axis**.
7. Place sales on the **y-axis**.

8. Give the chart a simple title.
9. Give both axes meaningful labels.
10. Display the chart.

Your goal is to practice the basic relationship:

DataFrame → Seaborn → Scatter Plot

Do not add advanced styling.

9. Concepts Used

For this exercise you will use:

- Python lists
- dictionaries
- Pandas
- DataFrames
- column names
- Seaborn
- `scatterplot()`
- `data`
- x-axis
- y-axis
- Matplotlib title/labels
- displaying a chart

You are combining ideas from several previous days:

```
Python collections
  ↓
Pandas DataFrame
  ↓
Seaborn visualization
```

10. Thought Process

Before writing code, think through the problem.

Step 1: What data do I need?

Two numerical features:

```
advertising spend
sales
```

Step 2: How should I store them?

Because you recently learned Pandas, use a:

```
DataFrame
```

Step 3: What am I trying to discover visually?

You want to see how:

```
advertising spend
```

relates to:

```
sales
```

Step 4: Which chart is suitable?

Both variables are numerical.

A simple:

```
scatter plot
```

is a good choice.

Step 5: What belongs on each axis?

Horizontal:

```
x = advertising spend
```

Vertical:

```
y = sales
```

Step 6: How will someone understand the chart?

Add:

```
title  
x-axis label  
y-axis label
```

Then display it.

11. Pseudocode

```
START

IMPORT Pandas
IMPORT Seaborn
IMPORT Matplotlib plotting tools

CREATE advertising spend values
CREATE corresponding sales values

STORE both columns in a dictionary

CREATE a DataFrame from the dictionary

CREATE a Seaborn scatter plot

SET advertising spend as x-axis data
SET sales as y-axis data

ADD a meaningful chart title
ADD x-axis label
ADD y-axis label

DISPLAY the chart

END
```

Notice that this describes **what the program should do**, without giving you the complete Python solution.

12. Suggested Solving Approach: Seaborn

Use this basic structure:

1. Prepare the data
↓
2. Create DataFrame
↓
3. Choose scatter plot
↓
4. Pass DataFrame to Seaborn
↓
5. Select x column
↓
6. Select y column
↓

7. Add title and labels

↓

8. Show chart

The most important Seaborn pattern to remember today is:

```
sns.plot_function(  
    data=your_dataframe,  
    x="column_name",  
    y="another_column"  
)
```

For today's problem, decide which Seaborn plotting function belongs in place of `plot_function`.

13. Easy Edge Cases

Repeated values

Suppose advertising spend contains:

```
100  
200  
200  
300
```

Repeated values are allowed.

Two points may have the same x-position.

If both their x and y values are identical, the points may appear directly on top of each other.

This does **not** automatically mean your program is wrong.

Very small dataset

Suppose you only have:

```
2 or 3 rows
```

Seaborn can still create the scatter plot.

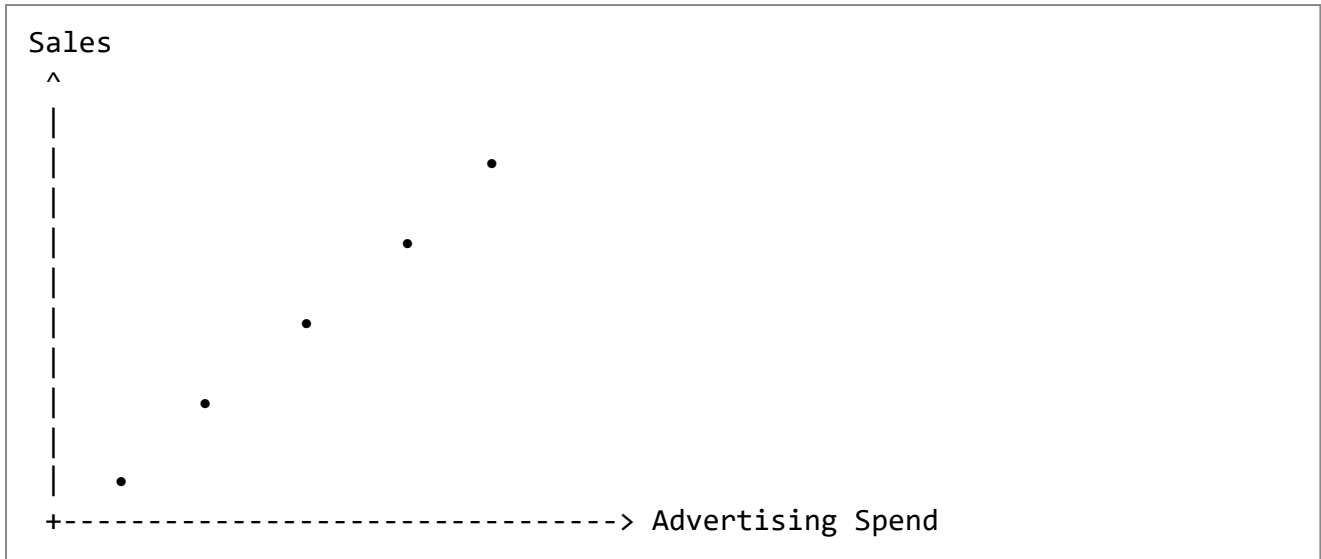
However, be careful about drawing strong conclusions from such a tiny amount of data.

For today's exercise, that is perfectly fine because the goal is learning how plotting works.

14. Expected Chart Description

You should see a chart containing several individual points.

Conceptually:



The horizontal axis should represent:

Advertising Spend

The vertical axis should represent:

Sales

If the sample values increase together, the points will generally move:

bottom-left → top-right

For this lesson, your goal is simply to **visualize that pattern**, not perform statistical analysis.

15. Hint Only

Start with these three libraries:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Create your DataFrame first.

Then think about filling in:

```
sns._____(
    data=_____,
    x="_____",
    y="_____"
)
```

Remember:

scatter plot → `sns.scatterplot()`

Then reuse what you learned yesterday about:

```
plt.title(...)  
plt.xlabel(...)  
plt.ylabel(...)  
plt.show()
```

Challenge: Try completing the program without looking up a full solution. The key new skill today is passing a **DataFrame** and its column names **directly to Seaborn**.